

09:00-09:50 (쉬는 시간 11:30-12:00)

좋은 리뷰는 취약보다 사용자 위험을 먼저 말합니다

P0~P3 severity와 actionable finding 기준을 합의한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

AGENTS.md
src/course_services/contracts.py

확인할 결과

review_rubric.md

3일차는 여덟 개 차시로 하나의 서비스를 완성합니다

오전 3개 차시 뒤 30분 휴식, 12:00-13:00 점심, 오후 5개 차시 뒤 휴식과 Q&A로 마무리합니다.

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	좋은 리뷰는 취향보다 사용자 위험을 먼저 말합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	review_rubric.md
09:50-10:40	2차시	unified diff를 읽어야 리뷰가 정확한 줄에 붙습니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	parsed_hunks.json
10:40-11:30	3차시	큰 저장소도 리뷰 문맥은 작고 명확해야 합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	context_pack.json
13:00-13:50	4차시	리뷰 결과도 API처럼 고정된 계약이 필요합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	review_report.json
13:50-14:40	5차시	Prompt baseline은 작은 고정 입력에서 출발합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	baseline_review.json
14:40-15:30	6차시	정적 분석과 LLM은 서로 다른 일을 말아야 합니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	hybrid_review.json
15:30-16:20	7차시	리뷰 Agent는 finding 수가 아니라 precision과 recall로 봅니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	review_eval.json
16:20-17:10	8차시	리뷰 개선은 prompt 감각이 아니라 오류 분석으로 끝냅니다	강의 12 시연 10 소프트웨어 실습 23 확인 5분	day3_review_report.md

11:30-12:00 쉬는 시간 · 12:00-13:00 점심시간 · 17:10-17:40 쉬는 시간 · 17:40-18:00 Q&A·실행 복구

스타일 지적이 많아질수록 중요한 correctness 오류가 묻힌다.

P0~P3 severity와 actionable finding 기준을 합의한다.

이번 차시의 확인 결과: review_rubric.md

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — P0~P3 severity와 actionable finding 기준을 합의한다.
- 02 — 모든 의견을 P1으로 부풀려 개발자가 경고를 무시한다. 왜 위험한지 설명한다.
- 03 — review_rubric.md에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Finding	한 가지 문제와 교정 제안
Severity	사용자 영향과 긴급도
False positive	문제가 아닌데 문제라고 한 결과
Confidence	근거에 대한 확신 정도

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

AGENTS.md

OUTPUT

review_rubric.md

PROCESS

변경 확인 → 사용자 영향 → 재현 조건

VERIFY

한 finding에는 path·line·severity·근거·제안이 모두 있다.
변경과 무관한 파일 의견은 게시 후보에서 제외한다.

실행 흐름은 다섯 단계로 고정합니다

01

변경 확인

02

사용자 영향

03

재현 조건

04

근거 라인

05

가장 작은 교정

마지막 판단은 review_rubric.md에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 ——— 한 finding에는 path·line·severity·근거·제안이 모두 있다.
- 경계 ——— 변경과 무관한 파일 의견은 게시 후보에서 제외한다.
- 외부 쓰기 ——— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 ——— status·error_code·입력 식별자·review_rubric.md

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

AGENTS.md

구현 코드

src/course_services/contracts.py

검증·test

data/day3_review_cases/unsafe_pr.diff

따라하기 notebook

materials/day3/day3_service_lab.ipynb

좋은 리뷰는 취향보다 사용자 위험을 먼저 말합니다

스타일 지적이 많아질수록 중요한 correctness 오류가 묻힌다.

correctness·security·data loss·contract break에 집중하고 style은 linter에 맡긴다.

이 차시에서
버릴 오해

모든 의견을 P1으로
부풀려 개발자가 경고를
무시한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Finding	한 가지 문제와 교정 제안	결정론 test로 먼저 확인
Severity	사용자 영향과 긴급도	adapter 경계를 확인
False positive	문제가 아닌데 문제라고 한 결과	비용·지연·권한을 확인
Confidence	근거에 대한 확신 정도	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

모든 의견을 P1으로 부풀려 개발자가
경고를 무시한다.

사용자 영향

스타일 지적이 많아질수록 중요한
correctness 오류가 묻힌다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: 변경과 무관한 파일 의견은 게시 후보에서 제외한다.
- 02 — 중단: 모든 의견을 P1으로 부풀려 개발자가 경고를 무시한다.
- 03 — 복구: correctness·security·data loss·contract break에 집중하고 style은 linter에 맡긴다.
- 04 — 증거: review_rubric.md과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

AGENTS.md 리뷰 기준을 읽고
바뀐 라인에만 finding을 작성하게
한다.

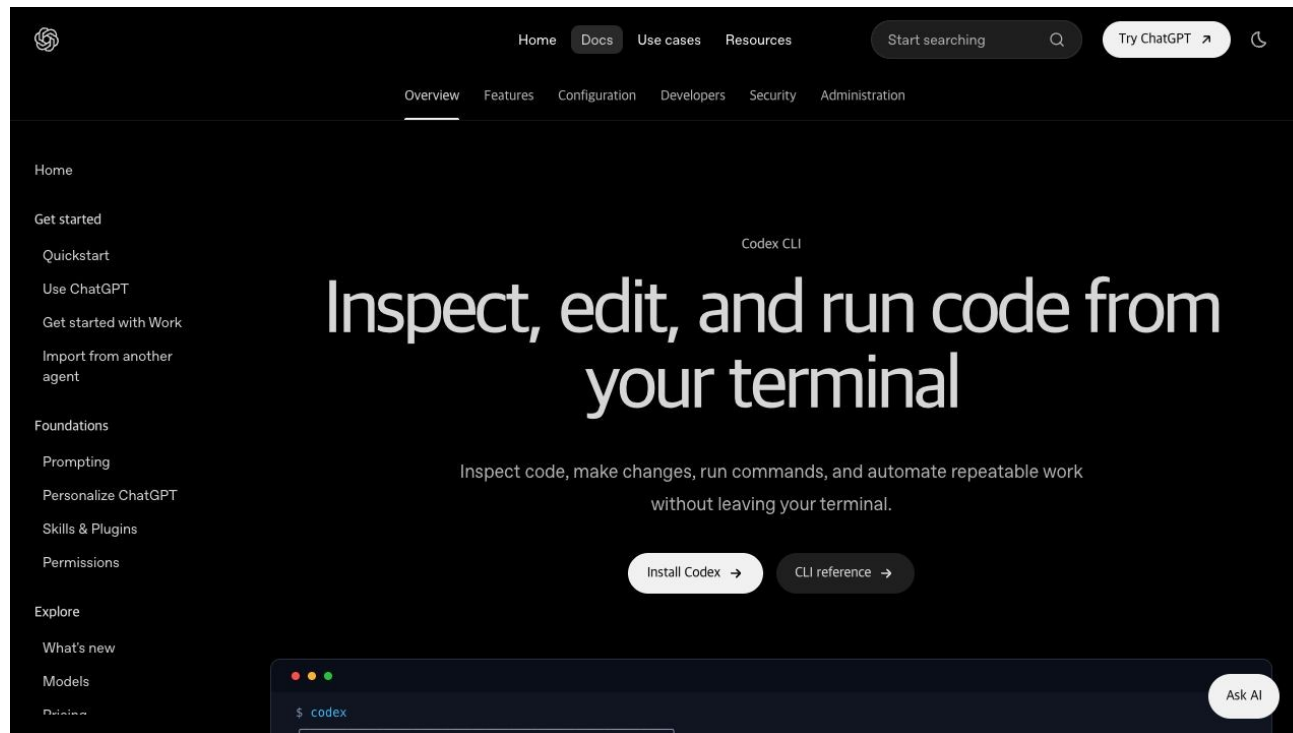
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

AGENTS.md 리뷰 기준을 읽고 바뀐 라인에만 finding을 작성하게 한다.

허용 경로

src/course_services/contracts.py
data/day3_review_cases/unsafe_pr.diff

완료 조건

한 finding에는 path·line·severity·근거·제안이 모두 있다.
변경과 무관한 파일 의견은 게시 후보에서 제외한다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 한 finding에는 path·line·severity·근거·제안이 모두 있다.
- 03 **Boundary test** — 변경과 무관한 파일 의견은 게시 후보에서 제외한다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: AGENTS.md
- 02 — 구현 확인: `src/course_services/contracts.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k maps_findings`
- 04 — 성공 신호: 한 finding에는 `path·line·severity·근거·제안`이 모두 있다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k maps_findings
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

review_rubric.md

사람이 확인할 세 줄

1. 한 finding에는 path·line·severity·근거·제안이 모두 있다.
2. 변경과 무관한 파일 의견은 게시 후보에서 제외한다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

모든 의견을 P1으로 부풀려 개발자가
경고를 무시한다.

복구

correctness·security·data
loss·contract break에 집중하고 style
은 linter에 맡긴다.

확인: 변경과 무관한 파일 의견은 게시 후보에서 제외한다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day3/day3_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

한 finding에는 path·line·severity·근거·제안이 모두 있다.
review_rubric.md 저장

STEP 1 · 입력과 설정을 확인합니다

파일: AGENTS.md

변수: 실행 경로·provider·dry-run 여부

결과 파일: review_rubric.md

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k maps_findings
```

성공 신호: 한 finding에는 path·line·severity·근거·제안이 모두 있다.

결과 파일: review_rubric.md

STEP 3 · 경계값을 바꿔 실패를 확인합니다

변경과 무관한 파일 의견은 게시 후보에서 제외한다.

복구: correctness·security·data loss·contract break에 집중하고 style은 linter에 맡긴다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

한 finding에는 path·line·severity·근거·제안이 모두 있다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

변경과 무관한 파일 의견은 게시 후보에서 제외한다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

작은 팀의 PR 대기 시간을 줄이는 첫 번째 자동 검토

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_rubric.md을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

작은 팀의 PR 대기 시간을 줄이는 첫 번째 자동 검토

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — review_rubric.md과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 한 finding에는 path·line·severity·근거·제안이 모두 있다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

correctness·security·data loss·contract break에 집중하고 style은 linter에 맡긴다.

관측

review_rubric.md에서 어떤 status·error_code를 보는가

1차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 좋은 리뷰는 취향보다 사용자 위험을 먼저 말합니다. 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k maps_findings`
- 03 — 증거: `review_rubric.md`와 정상·실패 test 결과가 남아 있다.

다음 차시: unified diff를 읽어야 리뷰가 정확한 줄에 붙습니다

unified diff를 읽어야 리뷰가 정확한 줄에 붙습니다

hunk header로 새 파일 line number를 복원한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

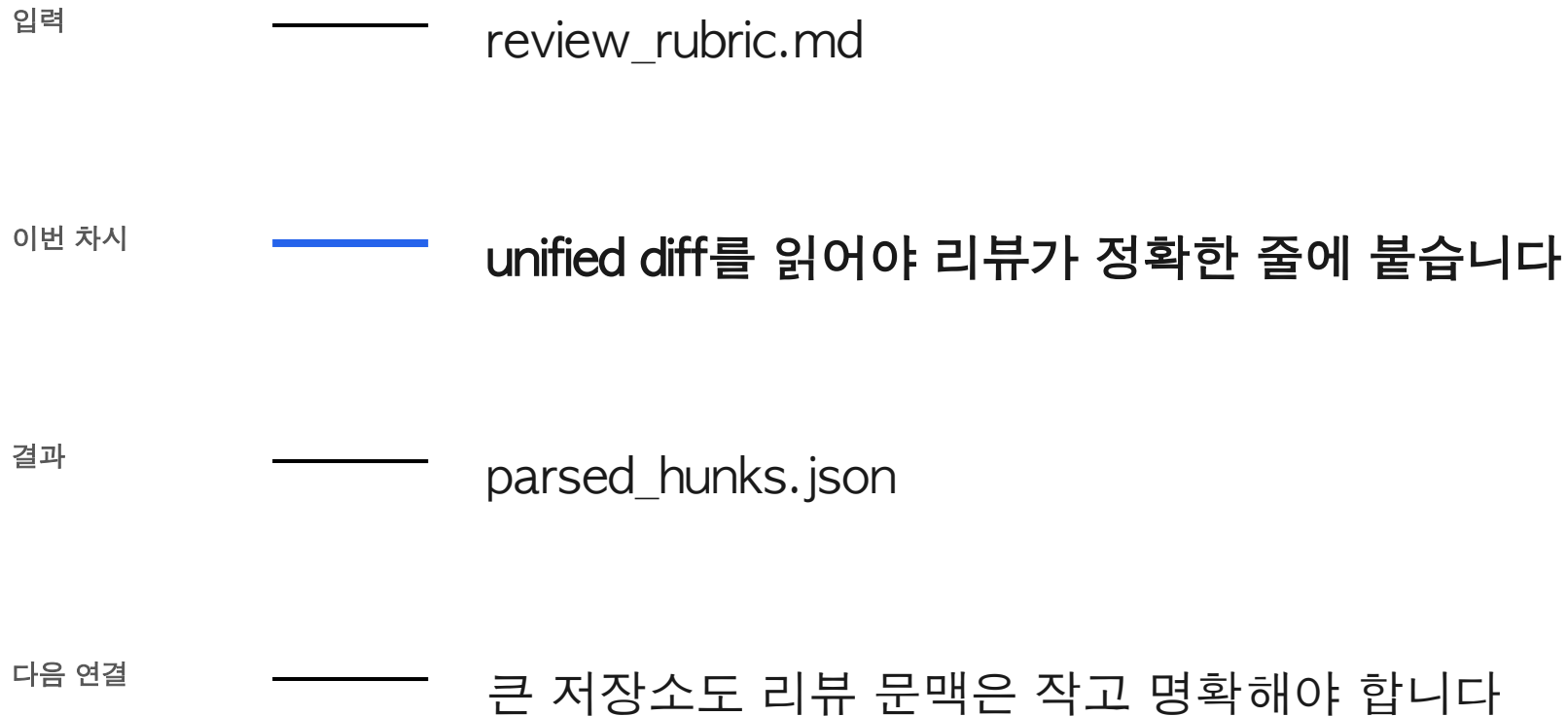
사용 파일

src/course_services/review_service.py
tests/test_course_services.py

확인할 결과

parsed_hunks.json

2차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



내용이 맞아도 line mapping이 틀리면 GitHub comment는 422 또는 엉뚱한 위치에 붙는다.

hunk header로 새 파일 line number를 복원한다.

이번 차시의 확인 결과: `parsed_hunks.json`

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — hunk header로 새 파일 line number를 복원한다.
- 02 — 파일 header와 코드의 + 기호를 같은 것으로 처리한다. 왜 위험한지 설명한다.
- 03 — parsed_hunks.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Diff	변경 전후 차이
Hunk	연속된 변경 구간
Old line	변경 전 파일의 줄
New line	변경 후 파일의 줄

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/course_services/review_service.py

OUTPUT

parsed_hunks.json

PROCESS

+++ target 확인 → @@ header parse → context는 양쪽 증가

VERIFY

추가된 9·11·12번 줄에 finding이 매핑된다.
absolute path와 hunk 없는 입력은 구조화 실패다.

실행 흐름은 다섯 단계로 고정합니다

01

+++ target 확인

02

@@ header parse

03

context는 양쪽
증가

04

삭제는 old만 증가

05

추가는 new에 기록

마지막 판단은 `parsed_hunks.json`에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	——	추가된 9·11·12번 줄에 finding이 매핑된다.
경계	——	absolute path와 hunk 없는 입력은 구조화 실패다.
외부 쓰기	——	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	——	status·error_code·입력 식별자·parsed_hunks.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅니다

입력·fixture

```
src/course_services/review_service.py
```

구현 코드

```
tests/test_course_services.py
```

검증·test

```
data/day3_review_cases/unsafe_pr.diff
```

따라하기 notebook

```
materials/day3/day3_service_lab.ipynb
```

unified diff를 읽어야 리뷰가 정확한 줄에 붙습니다

내용이 맞아도 line mapping이 틀리면 GitHub comment는 422 또는 엉뚱한 위치에 붙는다.

+++ header를 먼저 분리하고 hunk 안의 추가 라인만 수집한다.

이 차시에서
버릴 오해

파일 header와 코드의 +
기호를 같은 것으로
처리한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Diff	변경 전후 차이	결정론 test로 먼저 확인
Hunk	연속된 변경 구간	adapter 경계를 확인
Old line	변경 전 파일의 줄	비용·지연·권한을 확인
New line	변경 후 파일의 줄	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

파일 header와 코드의 + 기호를 같은
것으로 처리한다.

사용자 영향

내용이 맞아도 line mapping이 틀리면 GitHub
comment는 422 또는 엉뚱한 위치에 붙는
다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: absolute path와 hunk 없는 입력은 구조화 실패다.
- 02 — 중단: 파일 header와 코드의 + 기호를 같은 것으로 처리한다.
- 03 — 복구: +++ header를 먼저 분리하고 hunk 안의 추가 라인만 수집한다.
- 04 — 증거: parsed_hunks.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

rename·new file·삭제 only diff를
포함한 parser boundary test를
제안받는다.

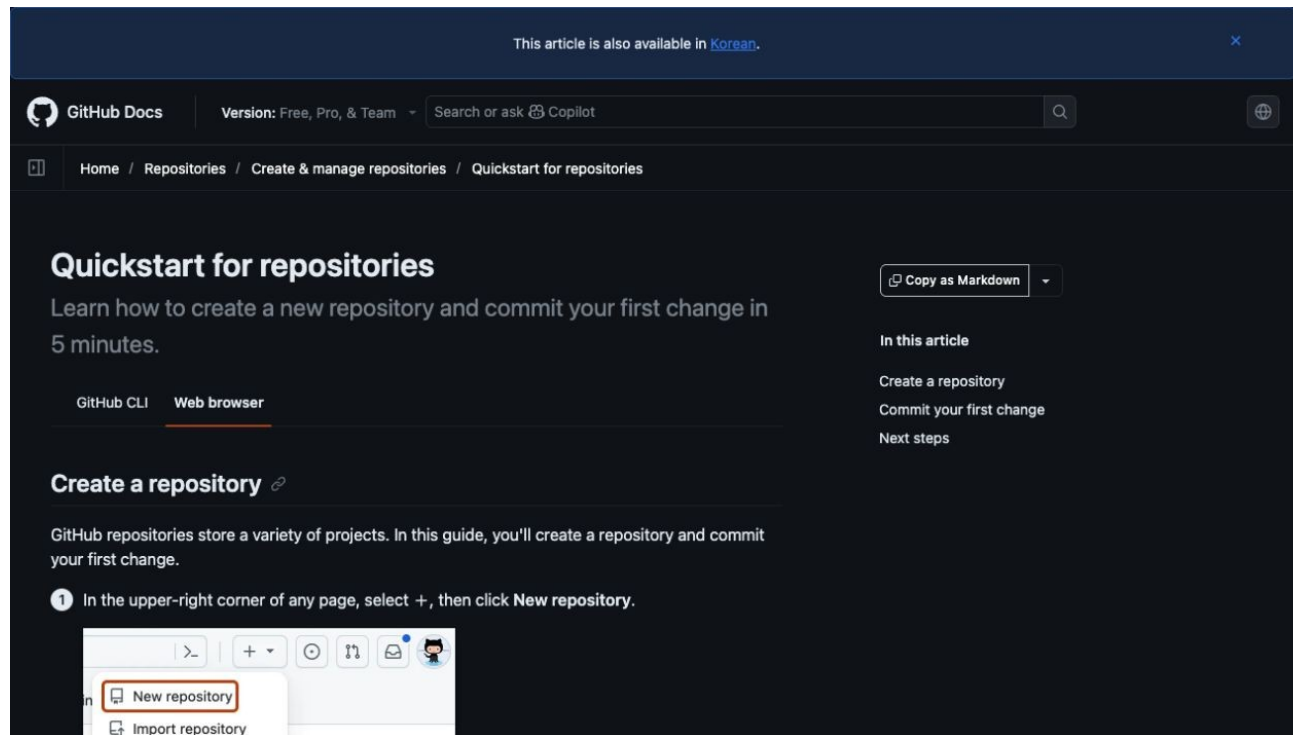
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

rename·new file·삭제 only diff를 포함한 parser boundary test를 제안받는다.

허용 경로

tests/test_course_services.py
data/day3_review_cases/unsafe_pr.diff

완료 조건

추가된 9·11·12번 줄에 finding이 매핑된다.
absolute path와 hunk 없는 입력은 구조화 실패다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 추가된 9·11·12번 줄에 finding이 매핑된다.
- 03 **Boundary test** — absolute path와 hunk 없는 입력은 구조화 실패다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/review_service.py`
- 02 — 구현 확인: `tests/test_course_services.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k unified_diff`
- 04 — 성공 신호: 추가된 9·11·12번 줄에 `finding0`이 매핑된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k unified_diff
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

`parsed_hunks.json`

사람이 확인할 세 줄

1. 추가된 9·11·12번 줄에 finding이 매핑된다.
2. absolute path와 hunk 없는 입력은 구조화 실패다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

파일 header와 코드의 + 기호를 같은
것으로 처리한다.

복구

+++ header를 먼저 분리하고 hunk
안의 추가 라인만 수집한다.

확인: absolute path와 hunk 없는 입력은 구조화 실패다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day3/day3_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

추가된 9·11·12번 줄에 finding이 매핑된다.
parsed_hunks.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/course_services/review_service.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `parsed_hunks.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k unified_diff
```

성공 신호: 추가된 9·11·12번 줄에 finding이 매핑된다.

결과 파일: parsed_hunks.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

absolute path와 hunk 없는 입력은 구조화 실패다.

복구: +++ header를 먼저 분리하고 hunk 안의 추가 라인만 수집한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

추가된 9·11·12번 줄에 finding이 매핑된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

absolute path와 hunk 없는 입력은 구조화 실패다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

GitHub·GitLab에서 공통으로 쓰는 diff 분석 core

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — parsed_hunks.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

GitHub·GitLab에서 공통으로 쓰는 diff 분석 core

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — parsed_hunks.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 추가된 9·11·12번 줄에 finding이 매핑된다.
로 정의하는가

복구

+++ header를 먼저 분리하고 hunk 안의 추가 라인만
수집한다.

안전

어디에서 사람 승인과 external_write=false를
확인하는가

관측

parsed_hunks.json에서 어떤 status_error_code를
보는가

2차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: unified diff를 읽어야 리뷰가 정확한 줄에 붙습니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k unified_diff`
- 03 — 증거: `parsed_hunks.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 큰 저장소도 리뷰 문맥은 작고 명확해야 합니다

10:40-11:30 (쉬는 시간 11:30-12:00)

큰 저장소도 리뷰 문맥은 작고 명확해야 합니다

변경 파일·직접 호출부·관련 test만 context pack으로 고른다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

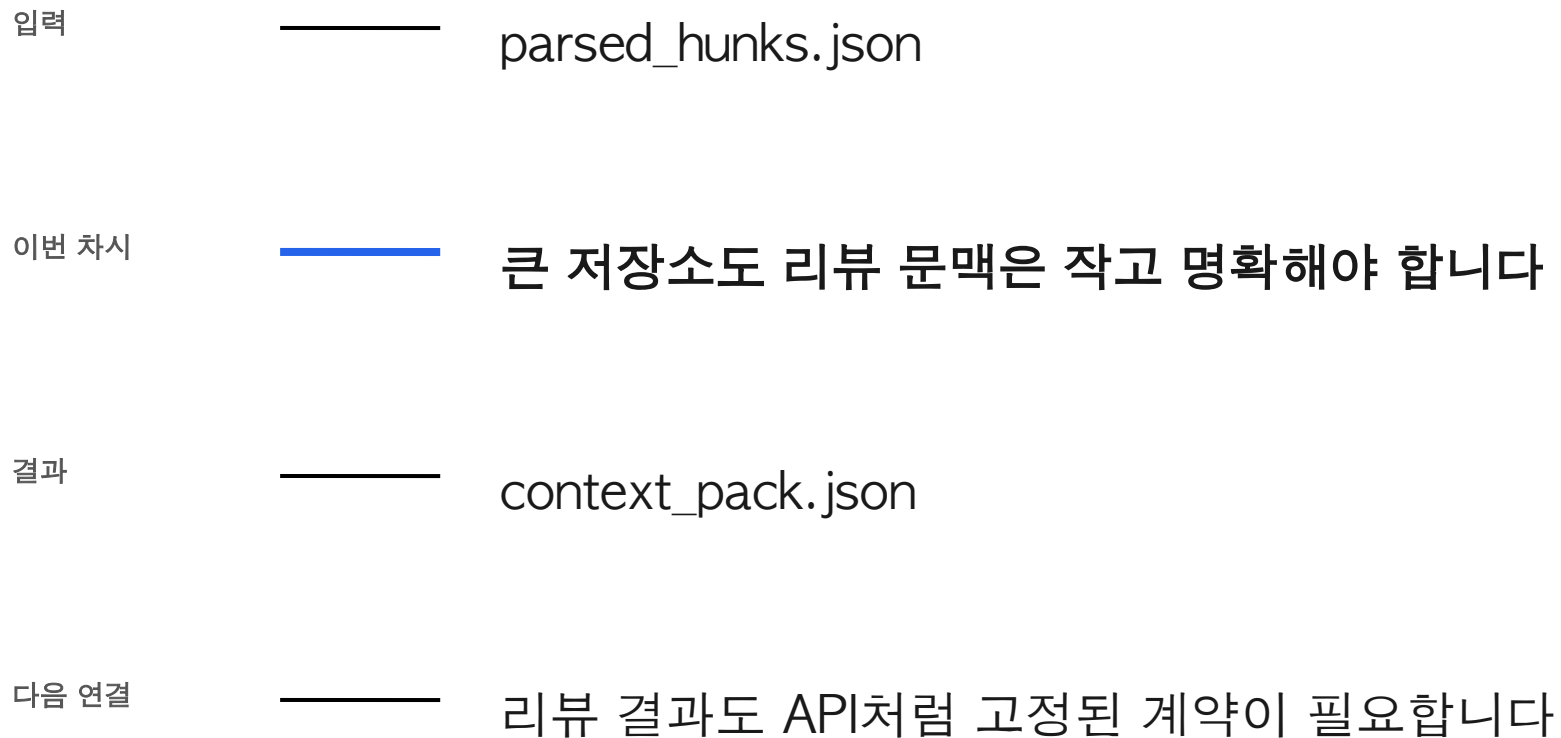
사용 파일

AGENTS.md
src/course_services/review_service.py

확인할 결과

context_pack.json

3차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



파일을 많이 넣으면 정확도가 자동으로 오르는 대신 비용과 상충 정보가 늘어난다.

변경 파일·직접 호출부·관련 test만 context pack으로 고른다.

이번 차시의 확인 결과: context_pack.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — 변경 파일·직접 호출부·관련 test만 context pack으로 고른다.
- 02 — 비밀키·대용량 로그·무관한 output까지 context에 넣는다. 왜 위험한지 설명한다.
- 03 — context_pack.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Repo context	변경을 이해하는 데 필요한 저장소 정보
Dependency	변경 코드가 의존하는 대상
Call site	함수를 실제 호출하는 위치
Context budget	한 번에 전달할 문맥 한도

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

AGENTS.md

OUTPUT

context_pack.json

PROCESS

changed path → 관련 symbol → 가까운 test

VERIFY

context pack에 changed_paths와 test_candidates가 남는다.
workspace 밖·.env·실행 cache는 context에서 제외한다.

실행 흐름은 다섯 단계로 고정합니다

01

changed path

02

관련 symbol

03

가까운 test

04

정책 문서

05

불필요 문맥 제외

마지막 판단은 context_pack.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— context pack에 changed_paths와 test_candidates가 남는다.
경계	—— workspace 밖·.env·실행 cache는 context에서 제외한다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·context_pack.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

AGENTS.md

구현 코드

src/course_services/review_service.py

검증·test

tests/test_course_services.py

따라하기 notebook

README.md

큰 저장소도 리뷰 문맥은 작고 명확해야 합니다

파일을 많이 넣으면 정확도가 자동으로 오르는 대신
비용과 상충 정보가 늘어난다.

rg로 후보를 좁히고 secret:generated output을 명시적으로 제외한다.

이 차시에서
버릴 오해

비밀키·대용량 로그·
무관한 output까지
context에 넣는다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Repo context	변경을 이해하는 데 필요한 저장소 정보	결정론 test로 먼저 확인
Dependency	변경 코드가 의존하는 대상	adapter 경계를 확인
Call site	함수를 실제 호출하는 위치	비용·지연·권한을 확인
Context budget	한 번에 전달할 문맥 한도	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

비밀키·대용량 로그·무관한 output
까지 context에 넣는다.

사용자 영향

파일을 많이 넣으면 정확도가 자동으로
오르는 대신 비용과 상충 정보가 늘어난다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: workspace 밖·.env·실행 cache는 context에서 제외한다.
- 02 — 중단: 비밀키·대용량 로그·무관한 output까지 context에 넣는다.
- 03 — 복구: rg로 후보를 좁히고 secret·generated output을 명시적으로 제외한다.
- 04 — 증거: context_pack.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

저장소를 이해하되 수정 전 관련
파일과 test를 먼저 나열하게
한다.

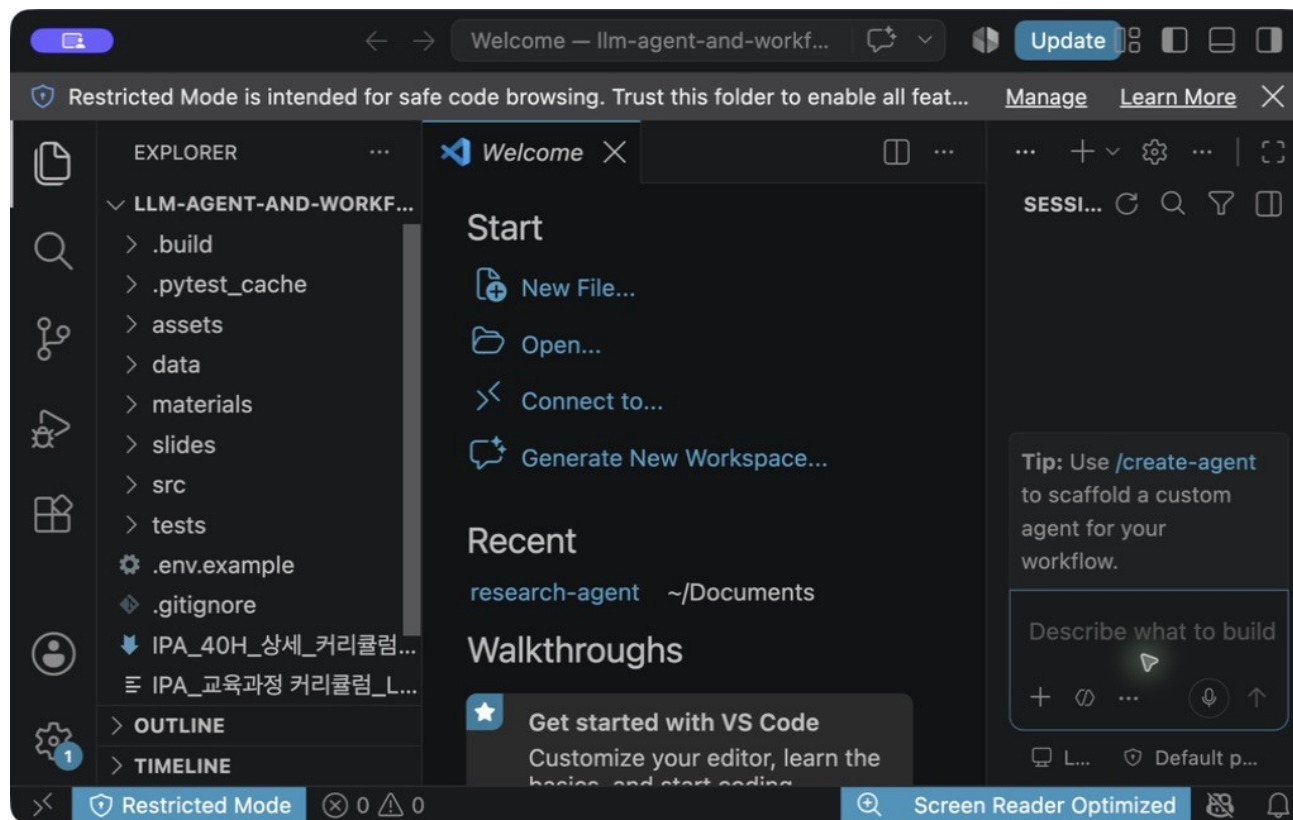
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

저장소를 이해하되 수정 전 관련 파일과 test를 먼저 나열하게 한다.

허용 경로

src/course_services/review_service.py
tests/test_course_services.py

완료 조건

context pack에 changed_paths와 test_candidates가 남는다.
workspace 밖·env·실행 cache는 context에서 제외한다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — context pack에 changed_paths와 test_candidates가 남는다.
- 03 **Boundary test** — workspace 밖·.env·실행 cache는 context에서 제외한다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: AGENTS.md
- 02 — 구현 확인: `src/course_services/review_service.py`
- 03 — 실행: `rg -n "run_review_service|ReviewFinding" src tests`
- 04 — 성공 신호: context pack에 `changed_paths`와 `test_candidates`가 남는다.

같은 저장소에서 이 명령을 실행합니다

```
$ rg -n "run_review_service|ReviewFinding" src tests
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

context_pack.json

사람이 확인할 세 줄

1. context pack에 changed_paths와 test_candidates가 남는다.
2. workspace 밖·.env·실행 cache는 context에서 제외한다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

비밀키·대용량 로그·무관한 output까지
context에 넣는다.

복구

rg로 후보를 좁히고 secret-generated
output을 명시적으로 제외한다.

확인: workspace 밖·.env·실행 cache는 context에서 제외한다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

README.md

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

context pack에 changed_paths와 test_candidates가 남는다.
context_pack.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: AGENTS.md

변수: 실행 경로·provider·dry-run 여부

결과 파일: context_pack.json

STEP 2 · 정상 경로를 실행합니다

```
rg -n "run_review_service|ReviewFinding" src tests
```

성공 신호: context pack에 changed_paths와 test_candidates가 남는다.

결과 파일: context_pack.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

workspace 밖·.env·실행 cache는 context에서 제외한다.
복구: rg로 후보를 좁히고 secret·generated output을 명시적으로 제외한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

context pack에 changed_paths와 test_candidates가 남는다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

workspace 밖·env·실행 cache는 context에서 제외한다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

수천 파일 monorepo에서 리뷰 latency와 비용을 줄이는 selection

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — context_pack.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

수천 파일 monorepo에서 리뷰 latency와 비용을 줄이는 selection

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — context_pack.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 context pack에 changed_paths와 test_candidates가 남는다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

rg로 후보를 좁히고 secret-generated output을 명시적으로 제외한다.

관측

context_pack.json에서 어떤 status_error_code를 보는가

3차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 큰 저장소도 리뷰 문맥은 작고 명확해야 합니다. 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `rg -n "run_review_service|ReviewFinding" src tests`
- 03 — 증거: `context_pack.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 리뷰 결과도 API처럼 고정된 계약이 필요합니다

13:00-13:50 (쉬는 시간 17:10-17:40)

리뷰 결과도 API처럼 고정된 계약이 필요합니다

ReviewFinding과 ReviewReport를 Pydantic으로 검증한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

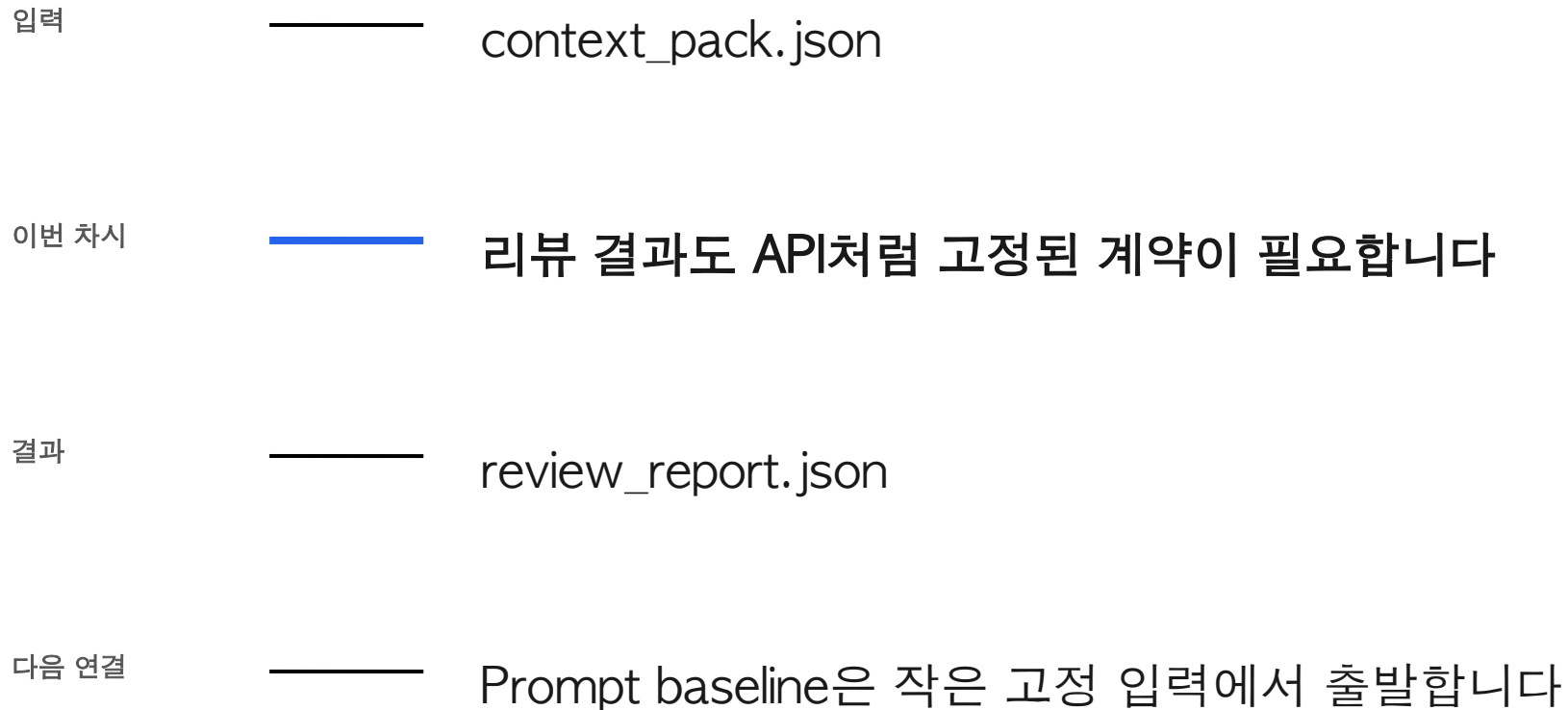
사용 파일

src/course_services/contracts.py
src/course_services/review_service.py

확인할 결과

review_report.json

4차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



자유 문장은 deduplication-line comment-평가에 다시 쓰기 어렵다.

ReviewFinding과 ReviewReport를 Pydantic으로 검증한다.

이번 차시의 확인 결과: review_report.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — ReviewFinding과 ReviewReport를 Pydantic으로 검증한다.
- 02 — schema 오류가 raw traceback으로 notebook을 끝낸다. 왜 위험한지 설명한다.
- 03 — review_report.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Contract	호출자와 구현 사이의 약속
extra=forbid	모르는 필드를 거부
Error code	복구 판단에 쓰는 이름
Automatic publish	사람 없이 게시되는 동작

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/course_services/contracts.py

OUTPUT

review_report.json

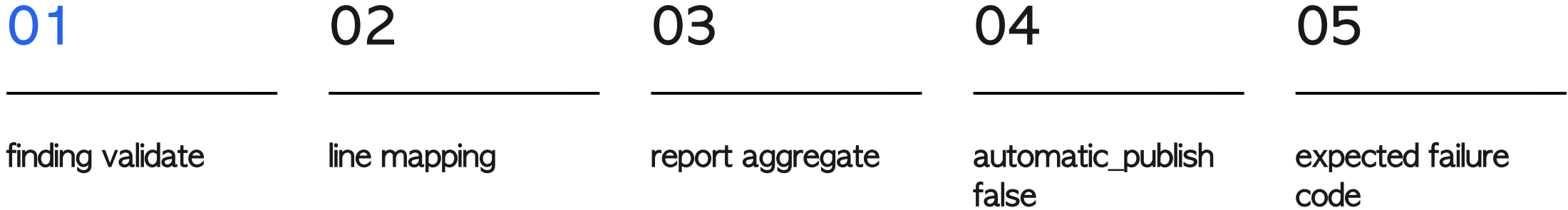
PROCESS

finding validate → line mapping → report aggregate

VERIFY

confidence 0~1 과 line>=1 이 검증된다.
automatic_publish=true는 model validation에서
차단된다.

실행 흐름은 다섯 단계로 고정합니다



마지막 판단은 review_report.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— confidence 0~1과 line>=1이 검증된다.
경계	—— automatic_publish=true는 model validation에서 차단된다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·review_report.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅니다

입력·fixture

```
src/course_services/contracts.py
```

구현 코드

```
src/course_services/review_service.py
```

검증·test

```
tests/test_course_services.py
```

따라하기 notebook

```
data/day5_eval/golden_review_findings.json
```

리뷰 결과도 API처럼 고정된 계약이 필요합니다

자유 문장은 deduplication-line comment·평가에 다시 쓰기 어렵다.

EXPECTED_FAILURE와 error_code로 바꾸고 원인은 trace에 남긴다.

이 차시에서
버릴 오해

schema 오류가 raw
traceback으로
notebook을 끝낸다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Contract	호출자와 구현 사이의 약속	결정론 test로 먼저 확인
extra=forbid	모르는 필드를 거부	adapter 경계를 확인
Error code	복구 판단에 쓰는 이름	비용·지연·권한을 확인
Automatic publish	사람 없이 게시되는 동작	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

schema 오류가 raw traceback으로
notebook을 끝낸다.

사용자 영향

자유 문장은 deduplication-line comment
평가에 다시 쓰기 어렵다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: `automatic_publish=true`는 `model validation`에서 차단된다.
- 02 — 중단: `schema` 오류가 `raw traceback`으로 `notebook`을 끝낸다.
- 03 — 복구: `EXPECTED_FAILURE`와 `error_code`로 바꾸고 원인은 `trace`에 남긴다.
- 04 — 증거: `review_report.json`과 `test` 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

contract 변경 시 정상·누락
field·extra field test를 함께
갱신한다.

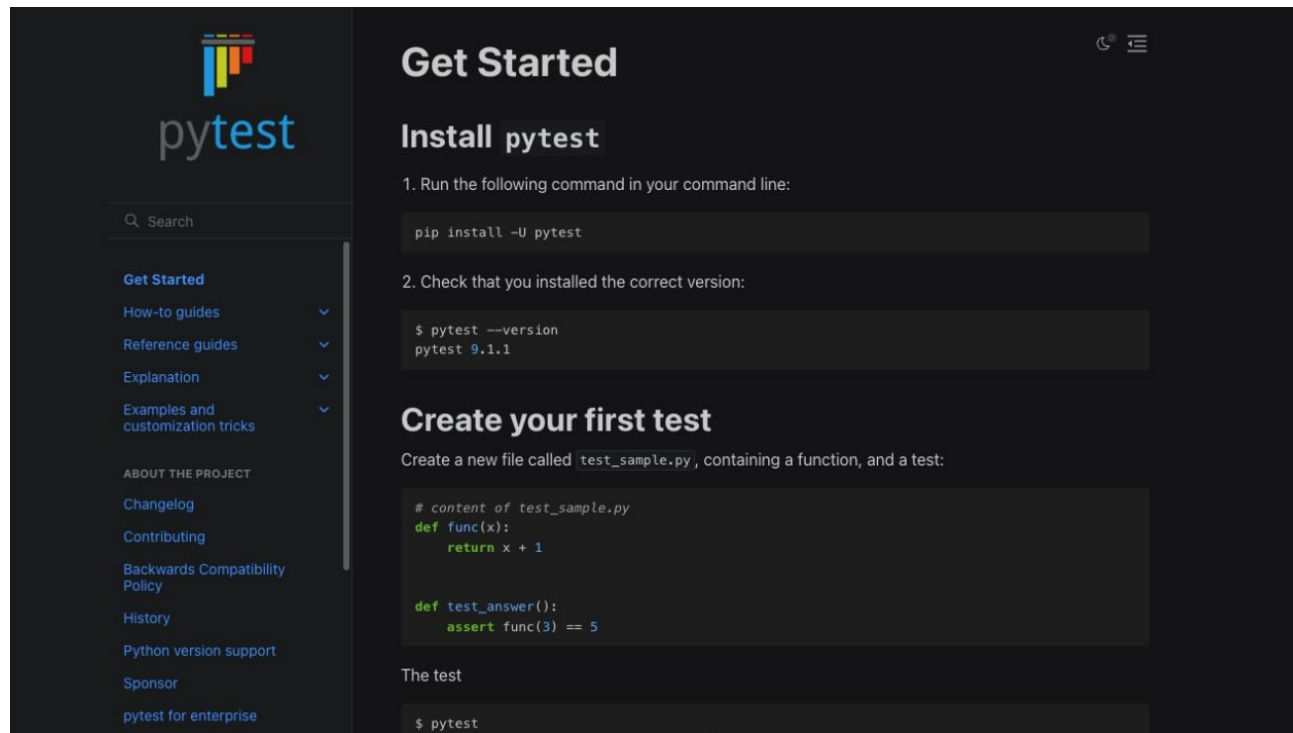
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

contract 변경 시 정상·누락 field·extra field test를 함께 갱신한다.

허용 경로

src/course_services/review_service.py
tests/test_course_services.py

완료 조건

confidence 0~1 과 line>=1 이 검증된다.
automatic_publish=true는 model validation에서 차단된다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — confidence 0~1 과 line>=1 이 검증된다.
- 03 **Boundary test** — automatic_publish=true는 model validation에서 차단된다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/contracts.py`
- 02 — 구현 확인: `src/course_services/review_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py`
- 04 — 성공 신호: `confidence 0~1` 과 `line>=1` 이 검증된다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

review_report.json

사람이 확인할 세 줄

1. confidence 0~1 과 line>=1 이 검증된다.
2. automatic_publish=true는 model validation 에서 차단된다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

schema 오류가 raw traceback으로 notebook을 끝낸다.

복구

EXPECTED_FAILURE와 error_code로 바꾸고 원인은 trace에 남긴다.

확인: automatic_publish=true는 model validation에서 차단된다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일	data/day5_eval/golden_review_findings.json
수정 범위	notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음
완료 기준	confidence 0~1 과 line>=1 이 검증된다. review_report.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/course_services/contracts.py`
변수: 실행 경로·`provider·dry-run` 여부

결과 파일: `review_report.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py
```

성공 신호: confidence 0~1 과 line>=1 이 검증된다.

결과 파일: review_report.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

`automatic_publish=true`는 model validation에서 차단된다.
복구: `EXPECTED_FAILURE`와 `error_code`로 바꾸고 원인은 trace에 남긴다.

결과에 `error_code`와 `external_write=false`가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

confidence 0~1 과 line>=1 이 검증된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

`automatic_publish=true`는 model validation에서 차단된다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

리뷰 결과를 IDE·PR·대시보드에서 같은 형식으로 재사용

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_report.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

리뷰 결과를 IDE·PR·대시보드에서 같은 형식으로 재사용

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — review_report.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 confidence 0~1과 line>=1이 검증된다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

EXPECTED_FAILURE와 error_code로 바꾸고 원인은 trace에 남긴다.

관측

review_report.json에서 어떤 status_error_code를 보는가

4차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 리뷰 결과도 API처럼 고정된 계약이 필요합니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: `review_report.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: Prompt baseline은 작은 고정 입력에서 출발합니다

Prompt baseline은 작은 고정 입력에서 출발합니다

의도적 bug diff에서 baseline finding을 만든다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

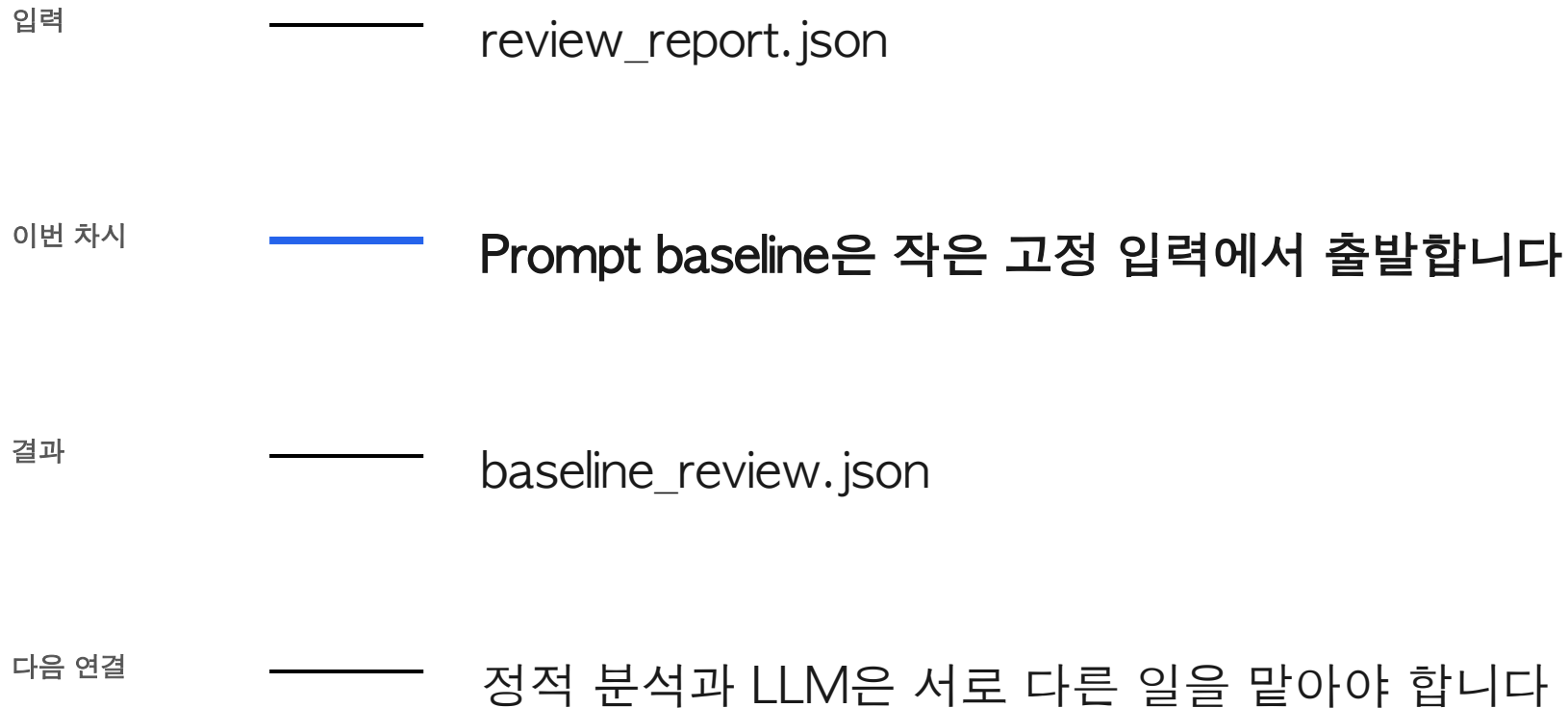
사용 파일

data/day3_review_cases/unsafe_pr.diff
src/course_services/review_service.py

확인할 결과

baseline_review.json

5차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



실제 PR만으로 튜닝하면 무엇이 개선됐는지 비교할 정답이 없다.

의도적 bug diff에서 baseline finding을 만든다.

이번 차시의 확인 결과: `baseline_review.json`

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — 의도적 bug diff에서 baseline finding을 만든다.
- 02 — LLM이 존재하지 않는 runtime 결과를 근거로 finding을 만든다. 왜 위험한지 설명한다.
- 03 — baseline_review.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Baseline	개선 전 비교 기준
Few-shot	입출력 예시를 prompt에 제공
Rule ID	같은 오류 유형을 묶는 식별자
Deterministic	같은 입력에 재현 가능한 결과

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

data/day3_review_cases/unsafe_pr.diff

PROCESS

synthetic diff → rule baseline → LLM candidate

OUTPUT

baseline_review.json

VERIFY

eval·외부 쓰기·broad except 세 finding이 나온다.
빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

실행 흐름은 다섯 단계로 고정합니다

01

synthetic diff

02

rule baseline

03

LLM candidate

04

schema validate

05

사람 비교

마지막 판단은 `baseline_review.json`에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 ——— eval·외부 쓰기·broad except 세 finding이 나온다.
- 경계 ——— 빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.
- 외부 쓰기 ——— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 ——— status·error_code·입력 식별자·baseline_review.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
data/day3_review_cases/unsafe_pr.diff
```

구현 코드

```
src/course_services/review_service.py
```

검증·test

```
materials/day3/day3_service_lab.ipynb
```

따라하기 notebook

```
output/day3-review/baseline.json
```

Prompt baseline은 작은 고정 입력에서 출발합니다

실제 PR만으로 튜닝하면 무엇이 개선됐는지 비교할
정답이 없다.

diff·static check·실제 test output만 evidence로 허용한다.

이 차시에서
버릴 오해

LLM이 존재하지 않는
runtime 결과를 근거로
finding을 만든다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Baseline	개선 전 비교 기준	결정론 test로 먼저 확인
Few-shot	입출력 예시를 prompt에 제공	adapter 경계를 확인
Rule ID	같은 오류 유형을 묶는 식별자	비용·지연·권한을 확인
Deterministic	같은 입력에 재현 가능한 결과	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

LLM이 존재하지 않는 runtime
결과를 근거로 finding을 만든다.

사용자 영향

실제 PR만으로 튜닝하면 무엇이 개선됐는지
비교할 정답이 없다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: 빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.
- 02 — 중단: LLM이 존재하지 않는 runtime 결과를 근거로 finding을 만든다.
- 03 — 복구: diff·static check·실제 test output만 evidence로 허용한다.
- 04 — 증거: baseline_review.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

baseline rule 하나를 구현하고
test가 먼저 실패하는지 확인한
뒤 수정한다.

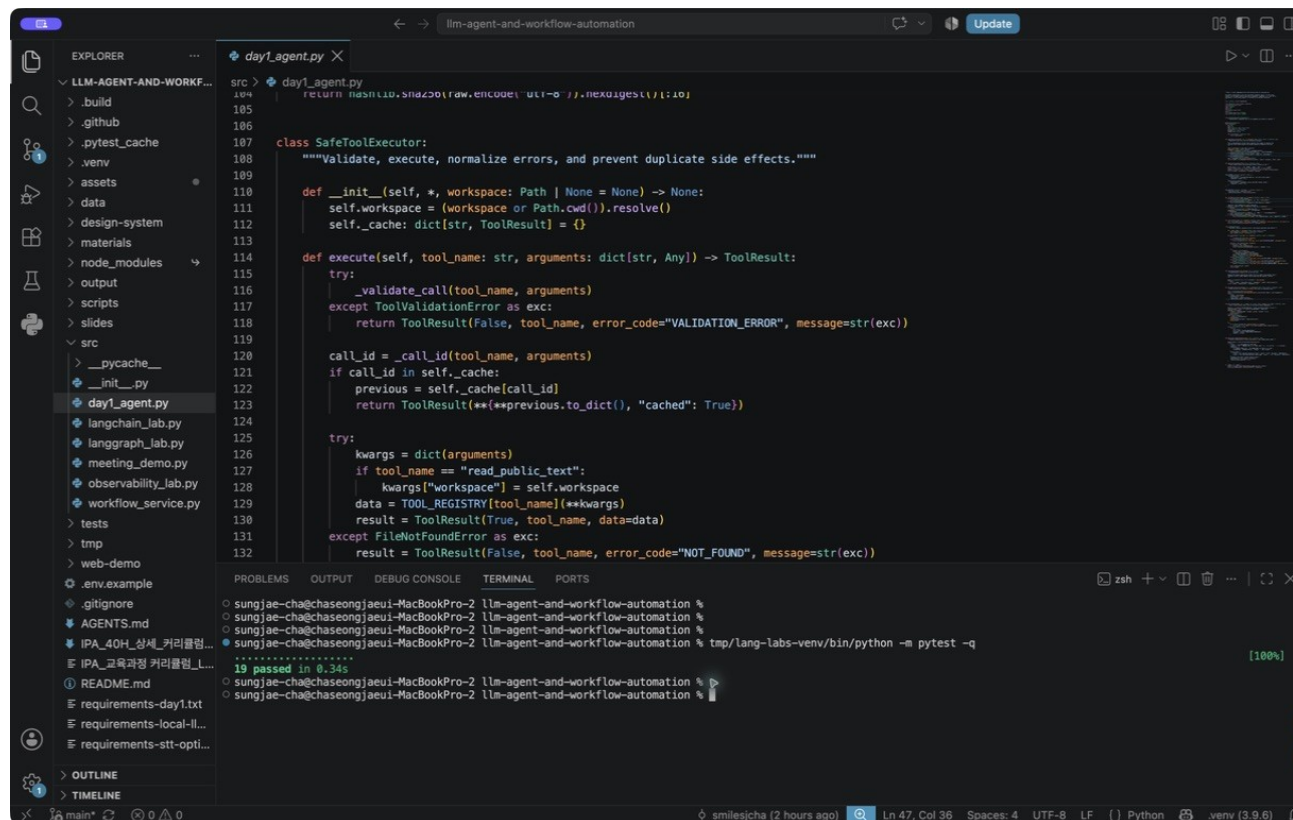
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

baseline rule 하나를 구현하고 test가 먼저 실패하는지 확인한 뒤 수정한다.

허용 경로

src/course_services/review_service.py
materials/day3/day3_service_lab.ipynb

완료 조건

eval·외부 쓰기·broad except 세 finding이 나온다.
빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — eval·외부 쓰기·broad except 세 finding이 나온다.
- 03 **Boundary test** — 빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `data/day3_review_cases/unsafe_pr.diff`
- 02 — 구현 확인: `src/course_services/review_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k maps_findings`
- 04 — 성공 신호: eval·외부 쓰기·broad except 세 finding이 나온다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k maps_findings
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

baseline_review.json

사람이 확인할 세 줄

1. eval·외부 쓰기·broad except 세 finding이 나온다.
2. 빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

LLM이 존재하지 않는 runtime 결과를
근거로 finding을 만든다.

복구

diff·static check·실제 test output만
evidence로 허용한다.

확인: 빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

output/day3-review/baseline.json

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

eval·외부 쓰기·broad except 세 finding이 나온다.
baseline_review.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: data/day3_review_cases/unsafe_pr.diff
변수: 실행 경로·provider·dry-run 여부

결과 파일: baseline_review.json

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k maps_findings
```

성공 신호: eval·외부 쓰기·broad except 세 finding이 나온다.

결과 파일: baseline_review.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.
복구: diff·static check·실제 test output만 evidence로 허용한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

eval·외부 쓰기·broad except 세 finding이 나온다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

빈 diff는 그럴듯한 빈 성공이 아니라 EMPTY_DIFF다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

교육용 bug repo에서 reviewer의 품질 변화 설명

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — baseline_review.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

교육용 bug repo에서 reviewer의 품질 변화 설명

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — baseline_review.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 eval·외부 쓰기·broad except 세 finding이 나온다.로 정의하는가

안전

어디에서 사람 승인과 external_write=false를 확인하는가

복구

diff·static check·실제 test output만 evidence로 허용한다.

관측

baseline_review.json에서 어떤 status·error_code를 보는가

5차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: Prompt baseline은 작은 고정 입력에서 출발합니다. 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k maps_findings`
- 03 — 증거: `baseline_review.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 정적 분석과 LLM은 서로 다른 일을 말아야 합니다

14:40-15:30 (쉬는 시간 17:10-17:40)

정적 분석과 LLM은 서로 다른 일을 맡아야 합니다

ruff·pytest 결과와 의미 리뷰를 한 context로 합친다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

AGENTS.md
tests/test_course_services.py

확인할 결과

hybrid_review.json

6차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	————	baseline_review.json
이번 차시	————	정적 분석과 LLM은 서로 다른 일을 맡아야 합니다
결과	————	hybrid_review.json
다음 연결	————	리뷰 Agent는 finding 수가 아니라 precision과 recall로 봅니다

문법·unused import까지 LLM에 맡기면 느리고 일관성도 낮다.

ruff·pytest 결과와 의미 리뷰를 한 context로 합친다.

이번 차시의 확인 결과: hybrid_review.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — ruff·pytest 결과와 의미 리뷰를 한 context로 합친다.
- 02 — 실행하지 않은 test를 passed라고 prompt에 써 둔다. 왜 위험한지 설명한다.
- 03 — hybrid_review.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Static analysis	코드를 실행하지 않고 규칙으로 검사
Unit test	작은 기능의 동작을 검증
Hybrid review	결정론 검사와 의미 검토 결합
Evidence	실제 명령과 변경 라인

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

AGENTS.md

OUTPUT

hybrid_review.json

PROCESS

ruff → focused pytest → diff parse

VERIFY

결정론 finding과 test evidence가 같은 report에 존재한다.
test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

실행 흐름은 다섯 단계로 고정합니다

01

ruff

02

focused pytest

03

diff parse

04

LLM review

05

deduplicate

마지막 판단은 hybrid_review.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

- 정상 ——— 결정론 finding과 test evidence가 같은 report에 존재한다.
- 경계 ——— test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.
- 외부 쓰기 ——— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
- 기록 ——— status·error_code·입력 식별자·hybrid_review.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅니다

입력·fixture

AGENTS.md

구현 코드

tests/test_course_services.py

검증·test

src/course_services/review_service.py

따라하기 notebook

.github/workflows/test.yml

정적 분석과 LLM은 서로 다른 일을 맡아야 합니다

문법·unused import까지 LLM에 맡기면 느리고 일관성도 낮다.

command·exit code·stdout 요약을 실행 후 evidence bundle에 넣는다.

이 차시에서
버릴 오해

실행하지 않은 test를
passed라고 prompt에
써 둔다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Static analysis	코드를 실행하지 않고 규칙으로 검사	결정론 test로 먼저 확인
Unit test	작은 기능의 동작을 검증	adapter 경계를 확인
Hybrid review	결정론 검사와 의미 검토 결합	비용·지연·권한을 확인
Evidence	실제 명령과 변경 라인	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

실행하지 않은 test를 passed라고
prompt에 써 둔다.

사용자 영향

문법·unused import까지 LLM에 맡기면
느리고 일관성도 낮다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.
- 02 — 중단: 실행하지 않은 test를 passed라고 prompt에 써 둔다.
- 03 — 복구: command·exit code·stdout 요약을 실행 후 evidence bundle에 넣는다.
- 04 — 증거: hybrid_review.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

코드를 만든 뒤 focused test와 full suite를 실제로 실행하고 결과를 보고하게 한다.

저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



vscode-pytest-25-passed-local.png

Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

코드를 만든 뒤 focused test와 full suite를 실제로 실행하고 결과를 보고하게 한다.

허용 경로

tests/test_course_services.py
src/course_services/review_service.py

완료 조건

결정론 finding과 test evidence가 같은 report에 존재한다.
test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- | | | | |
|----|---------------|----|--|
| 01 | Scope | —— | 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 결정론 finding과 test evidence가 같은 report에 존재한다. |
| 03 | Boundary test | —— | test 미실행은 READY_FOR_HUMAN_MERGE가 아니다. |
| 04 | Human merge | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다 |

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: AGENTS.md
- 02 — 구현 확인: tests/test_course_services.py
- 03 — 실행: python -m pytest -q tests/test_course_services.py
- 04 — 성공 신호: 결정론 finding과 test evidence가 같은 report에 존재한다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

hybrid_review.json

사람이 확인할 세 줄

1. 결정론 finding과 test evidence가 같은 report에 존재한다.
2. test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

실행하지 않은 test를 passed라고
prompt에 써 둔다.

복구

command·exit code·stdout 요약을
실행 후 evidence bundle에 넣는다.

확인: test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일 `.github/workflows/test.yml`

수정 범위 notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준 결정론 finding과 test evidence가 같은 report에 존재한다.
`hybrid_review.json` 저장

STEP 1 · 입력과 설정을 확인합니다

파일: AGENTS.md

변수: 실행 경로·provider·dry-run 여부

결과 파일: hybrid_review.json

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py
```

성공 신호: 결정론 finding과 test evidence가 같은 report에 존재한다.

결과 파일: hybrid_review.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.
복구: command·exit code·stdout 요약을 실행 후 evidence bundle에 넣는다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

결정론 finding과 test evidence가 같은 report에 존재한다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

test 미실행은 READY_FOR_HUMAN_MERGE가 아니다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

PR이 열리면 자동 검사 후 개발자가 고위험 finding만 검토

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — hybrid_review.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

PR이 열리면 자동 검사 후 개발자가 고위험 finding만 검토

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — hybrid_review.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 결정론 finding과 test evidence가 같은 report에 존재한다.로 정의하는가

복구

command-exit code-stdout 요약을 실행 후 evidence bundle에 넣는다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

hybrid_review.json에서 어떤 status-error_code를 보는가

6차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 정적 분석과 LLM은 서로 다른 일을 맡아야 합니다. 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: `hybrid_review.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 리뷰 Agent는 finding 수가 아니라 precision과 recall로 봅니다

15:30-16:20 (쉬는 시간 17:10-17:40)

리뷰 Agent는 finding 수가 아니라 precision 과 recall로 봅니다

golden finding과 predicted finding을 exact key로 비교한다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

사용 파일

src/course_services/eval_service.py
data/day5_eval/
golden_review_findings.json

확인할 결과

review_eval.json

7차시는 앞의 결과를 이어받아 다음 상태를 만듭니다

입력	hybrid_review.json
이번 차시	리뷰 Agent는 finding 수가 아니라 precision과 recall로 봅니다
결과	review_eval.json
다음 연결	리뷰 개선은 prompt 감각이 아니라 오류 분석으로 끝냅니다

많이 지적하는 reviewer가 실제로 더 좋은 reviewer라는 보장은 없다.

golden finding과 predicted finding을 exact key로 비교한다.

이번 차시의 확인 결과: review_eval.json

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — golden finding과 predicted finding을 exact key로 비교한다.
- 02 — 제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다. 왜 위험한지 설명한다.
- 03 — review_eval.json에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Precision	제기한 문제 중 맞은 비율
Recall	잡아야 할 문제 중 찾은 비율
F1	precision과 recall의 조화 평균
Golden set	합의된 기대 finding 모음

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/course_services/eval_service.py

OUTPUT

review_eval.json

PROCESS

key 정규화 → 교집합 → false positive

VERIFY

fixture case precision·recall·F1이 1.0이다.
recall 또는 precision 0.8 미만이면 HOLD다.

실행 흐름은 다섯 단계로 고정합니다

01

key 정규화

02

교집합

03

false positive

04

false negative

05

release gate

마지막 판단은 review_eval.json에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	—— fixture case precision·recall·F1이 1.0이다.
경계	—— recall 또는 precision 0.8 미만이면 HOLD다.
외부 쓰기	—— 기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	—— status·error_code·입력 식별자·review_eval.json

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/course_services/eval_service.py
```

구현 코드

```
data/day5_eval/golden_review_findings.json
```

검증·test

```
tests/test_course_services.py
```

따라하기 notebook

```
materials/day3/day3_service_lab.ipynb
```

리뷰 Agent는 finding 수가 아니라 precision과 recall로 봅니다

많이 지적하는 reviewer가 실제로 더 좋은 reviewer라는 보장은 없다.

path·line·rule_id 같은 안정 key를 평가에 사용한다.

이 차시에서
버릴 오해

제목 문구가 조금
달라졌다고 같은 finding
을 다른 것으로 센다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Precision	제기한 문제 중 맞은 비율	결정론 test로 먼저 확인
Recall	잡아야 할 문제 중 찾은 비율	adapter 경계를 확인
F1	precision과 recall의 조화 평균	비용·지연·권한을 확인
Golden set	합의된 기대 finding 모음	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다.

사용자 영향

많이 지적하는 reviewer가 실제로 더 좋은 reviewer라는 보장은 없다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: recall 또는 precision 0.8 미만이면 HOLD다.
- 02 — 중단: 제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다.
- 03 — 복구: path·line·rule_id 같은 안정 key를 평가에 사용한다.
- 04 — 증거: review_eval.json과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

평가 함수를 구현할 때 empty prediction·empty expected 경계를 test한다.

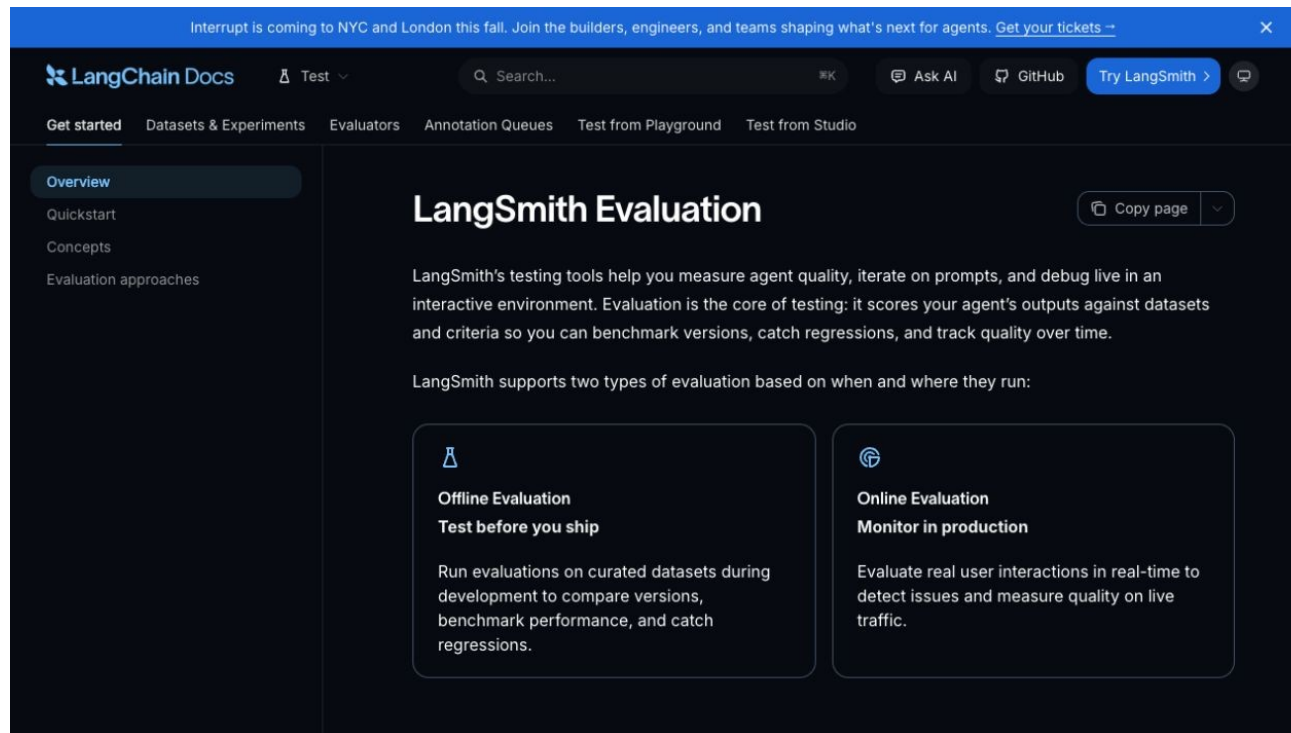
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

평가 함수를 구현할 때 empty prediction·empty expected 경계를 test한다.

허용 경로

data/day5_eval/golden_review_findings.json
tests/test_course_services.py

완료 조건

fixture case precision·recall·F1이 1.0이다.
recall 또는 precision 0.8 미만이면 HOLD다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — fixture case precision·recall·F1 이 1.0이다.
- 03 **Boundary test** — recall 또는 precision 0.8 미만이면 HOLD다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/eval_service.py`
- 02 — 구현 확인: `data/day5_eval/golden_review_findings.json`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k offline_eval`
- 04 — 성공 신호: fixture case precision·recall·F1이 1.00이다.

같은 저장소에서 이 명령을 실행합니다

```
$ python -m pytest -q tests/test_course_services.py -k offline_eval
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

review_eval.json

사람이 확인할 세 줄

1. fixture case precision·recall·F1이 1.0이다.
2. recall 또는 precision 0.8 미만이면 HOLD다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

제목 문구가 조금 달라졌다고 같은 finding을 다른 것으로 센다.

복구

path·line·rule_id 같은 안정 key를 평가에 사용한다.

확인: recall 또는 precision 0.8 미만이면 HOLD다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day3/day3_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

fixture case precision·recall·F1이 1.0이다.
review_eval.json 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/course_services/eval_service.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `review_eval.json`

STEP 2 · 정상 경로를 실행합니다

```
python -m pytest -q tests/test_course_services.py -k offline_eval
```

성공 신호: fixture case precision·recall·F1 이 1.0이다.

결과 파일: review_eval.json

STEP 3 · 경계값을 바꿔 실패를 확인합니다

recall 또는 precision 0.8 미만이면 HOLD다.
복구: path·line·rule_id 같은 안정 key를 평가에 사용한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

fixture case precision·recall·F1 이 1.0이다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

recall 또는 precision 0.8 미만이면 HOLD다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

모델 교체 전 기존 reviewer보다 나아졌는지 정량 비교

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_eval.json을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

모델 교체 전 기존 reviewer보다 나아졌는지 정량 비교

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — review_eval.json과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 fixture case precision-recall-F1이 1.0이다.로 정의하는가

복구

path·line·rule_id 같은 안정 key를 평가에 사용한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

review_eval.json에서 어떤 status·error_code를 보는가

7차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 리뷰 Agent는 finding 수가 아니라 precision과 recall로 봅니다. 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k offline_eval`
- 03 — 증거: `review_eval.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: 리뷰 개선은 prompt 감각이 아니라 오류 분석으로 끝납니다

16:20-17:10 (쉬는 시간 17:10-17:40)

리뷰 개선은 prompt 감각이 아니라 오류 분석으로 끝냅니다

false positive·false negative 원인을 분류하고 Day 3 report를 만든다.

진행

강의 12분 · 강사 시연 10분 · 소프트웨어
실습 23분 · 실행 확인 5분

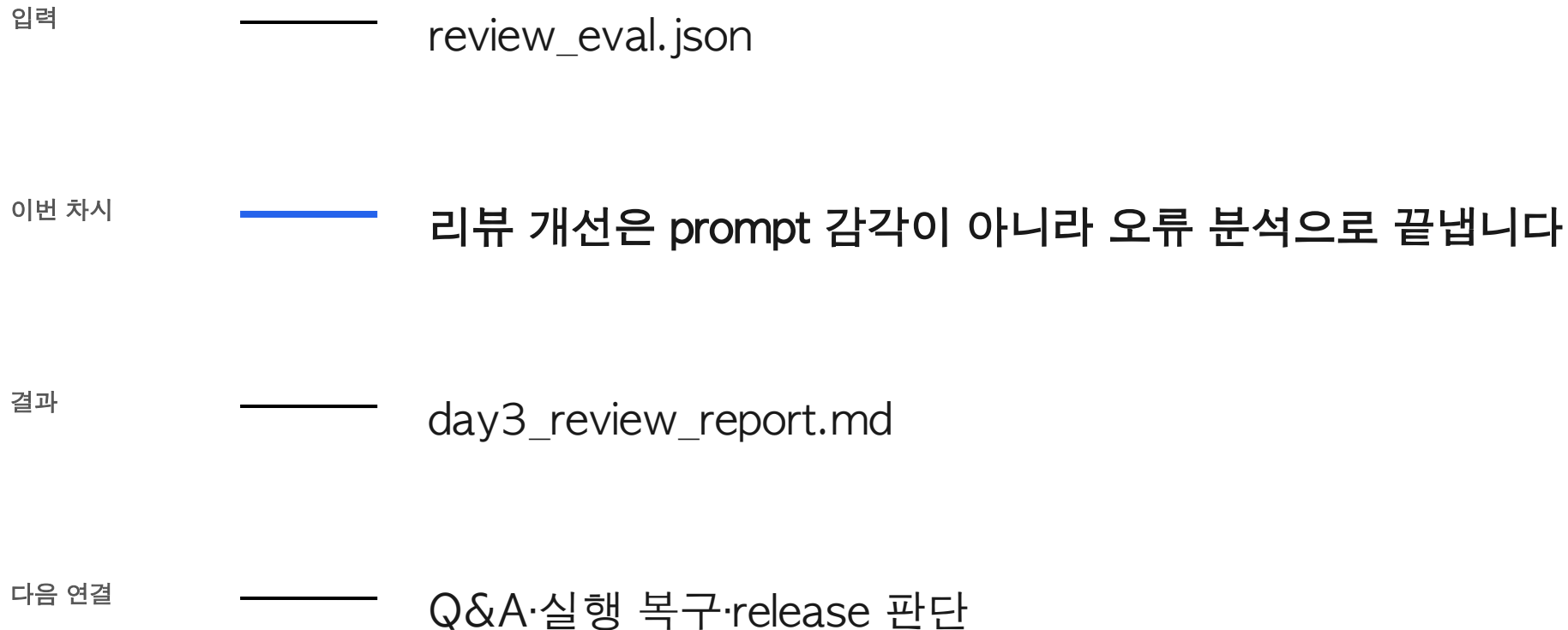
사용 파일

src/course_services/codex_harness.py
tests/test_course_services.py

확인할 결과

day3_review_report.md

8차시는 앞의 결과를 이어받아 다음 상태를 만듭니다



평균 점수만 보면 특정 언어·파일·위험 유형의 반복 실패가 보이지 않는다.

false positive·false negative 원인을 분류하고 Day 3 report를 만든다.

이번 차시의 확인 결과: day3_review_report.md

이 차시가 끝나면 세 가지를 설명하고 실행할 수 있습니다

- 01 — false positive·false negative 원인을 분류하고 Day 3 report를 만든다.
- 02 — 점수를 올리려고 기존 assertion이나 policy를 약화한다. 왜 위험한지 설명한다.
- 03 — day3_review_report.md에서 정상과 실패 상태를 직접 확인한다.

처음 보는 용어는 이 네 가지 역할만 구분합니다

용어	이 수업에서 쓰는 뜻
Error analysis	틀린 결과의 패턴을 분류
Regression	이전보다 나빠진 동작
Ablation	구성요소를 빼 영향 확인
Human merge	최종 반영은 사람이 결정

입력·처리·결과·검증을 분리하면 문제가 빨리 보입니다

INPUT

src/course_services/codex_harness.py

OUTPUT

day3_review_report.md

PROCESS

실패 목록 → 원인 label → 최소 수정

VERIFY

허용된 path·focused test·diff review가 모두 충족된다.
.env 변경·secret 탐지·미실행 test 중 하나라도 있으면
HOLD다.

실행 흐름은 다섯 단계로 고정합니다

01

실패 목록

02

원인 label

03

최소 수정

04

회귀 test

05

commit·tag

마지막 판단은 day3_review_report.md에서 사람이 확인합니다.

코드보다 먼저 성공·실패 계약을 정합니다

정상	허용된 path-focused test-diff review가 모두 충족된다.
경계	.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.
외부 쓰기	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	status·error_code·입력 식별자·day3_review_report.md

어떤 파일을 열고 어디에서 결과를 확인하는지 먼저 봅시다

입력·fixture

```
src/course_services/codex_harness.py
```

구현 코드

```
tests/test_course_services.py
```

검증·test

```
AGENTS.md
```

따라하기 notebook

```
materials/day3/day3_service_lab.ipynb
```

리뷰 개선은 prompt 감각이 아니라 오류 분석으로 끝냅니다

평균 점수만 보면 특정 언어·파일·위험 유형의 반복 실패가 보이지 않는다.

실패를 대표 test로 추가하고 최소 변경만 허용한다.

이 차시에서
버릴 오해

점수를 올리려고 기존
assertion이나 policy를
약화한다.

비슷해 보이는 선택도 책임과 검증 범위가 다릅니다

구분	역할	이번 차시의 판단 기준
Error analysis	틀린 결과의 패턴을 분류	결정론 test로 먼저 확인
Regression	이전보다 나빠진 동작	adapter 경계를 확인
Ablation	구성요소를 빼 영향 확인	비용·지연·권한을 확인
Human merge	최종 반영은 사람이 결정	사람 판단과 운영 기록을 확인

가장 먼저 재현할 실패는 이것입니다

실패

점수를 올리려고 기존 assertion이나 policy를 약화한다.

사용자 영향

평균 점수만 보면 특정 언어·파일·위험 유형의 반복 실패가 보이지 않는다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구는 재시도보다 먼저 중단 조건을 정합니다

- 01 — 탐지: .env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.
- 02 — 중단: 점수를 올리려고 기존 assertion이나 policy를 약화한다.
- 03 — 복구: 실패를 대표 test로 추가하고 최소 변경만 허용한다.
- 04 — 증거: day3_review_report.md과 test 결과를 함께 남긴다.

Codex는 코드를 대신 쓰는 도구보다 검증 루프에 가깝습니다

diff를 검토하고 actionable finding
만 보고한 뒤 test 증거를 다시
확인한다.

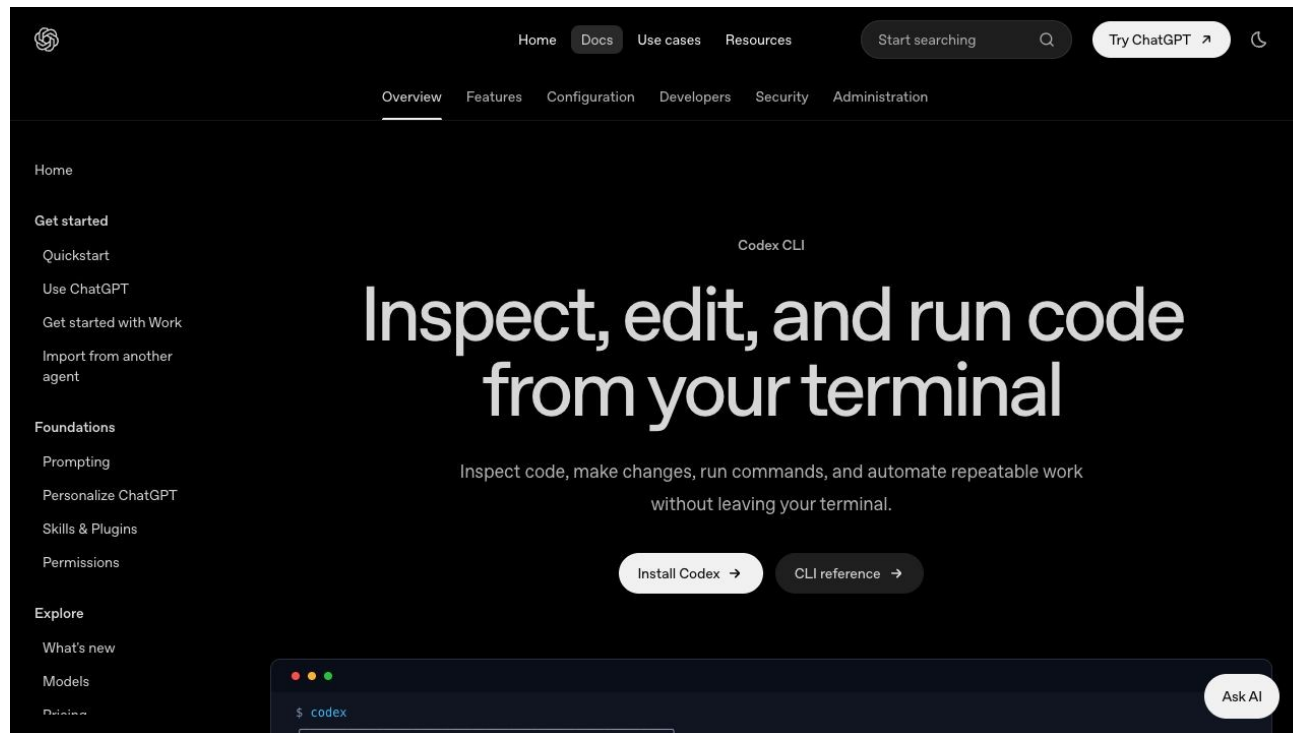
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex에게는 목표보다 완료 조건을 더 구체적으로 줍니다

목표

diff를 검토하고 actionable finding만 보고한 뒤 test 증거를 다시 확인한다.

허용 경로

tests/test_course_services.py
AGENTS.md

완료 조건

허용된 path-focused test-diff review가 모두 충족된다.
.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

생성된 patch는 diff와 test 증거를 보고 사람이 결정합니다

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 허용된 path·focused test·diff review가 모두 충족된다.
- 03 **Boundary test** — .env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사가 먼저 정상 경로를 끝까지 실행합니다

- 01 — 열기: `src/course_services/codex_harness.py`
- 02 — 구현 확인: `tests/test_course_services.py`
- 03 — 실행: `git diff -- src/course_services tests/test_course_services.py`
- 04 — 성공 신호: 허용된 `path·focused test·diff review`가 모두 충족된다.

같은 저장소에서 이 명령을 실행합니다

```
$ git diff -- src/course_services tests/test_course_services.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 여부는 화면이 아니라 결과 계약으로 확인합니다

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

day3_review_report.md

사람이 확인할 세 줄

1. 허용된 path-focused test-diff review가 모두 충족된다.
2. .env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.
3. external_write=false

강사는 실패 한 건도 바로 재현하고 복구합니다

재현

점수를 올리려고 기존 assertion이나 policy를 약화한다.

복구

실패를 대표 test로 추가하고 최소 변경만 허용한다.

확인: .env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.

이제 같은 코드를 각자 실행하고 한 줄만 바꿉니다

열 파일

materials/day3/day3_service_lab.ipynb

수정 범위

notebook의 실습 변수 또는 fixture 한 줄 · src 전체를 복사해 바꾸지 않음

완료 기준

허용된 path-focused test-diff review가 모두 충족된다.
day3_review_report.md 저장

STEP 1 · 입력과 설정을 확인합니다

파일: `src/course_services/codex_harness.py`
변수: 실행 경로·provider·dry-run 여부

결과 파일: `day3_review_report.md`

STEP 2 · 정상 경로를 실행합니다

```
git diff -- src/course_services tests/test_course_services.py
```

성공 신호: 허용된 path-focused test-diff review가 모두 충족된다.

결과 파일: day3_review_report.md

STEP 3 · 경계값을 바꿔 실패를 확인합니다

.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.
복구: 실패를 대표 test로 추가하고 최소 변경만 허용한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 case를 focused test로 확인합니다

NORMAL

허용된 path-focused test-diff review가 모두 충족된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

가장 중요한 실패 조건을 test로 고정합니다

BOUNDARY

.env 변경·secret 탐지·미실행 test 중 하나라도 있으면 HOLD다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

재직자는 작은 운영 문제 한 곳에 먼저 적용합니다

현업은 merge 보조, 구직자는 리뷰 품질 리포트와 회귀 test를 포트폴리오로 제출

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — day3_review_report.md을 운영 검토 자료로 사용

구직자는 제품 판단과 검증 증거를 함께 보여줍니다

현업은 merge 보조, 구직자는 리뷰 품질 리포트와 회귀 test를 포트폴리오로 제출

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — day3_review_report.md과 README 재현 절차를 제출

서비스로 운영하려면 이 네 가지 질문에 답해야 합니다

품질

무엇을 허용된 path-focused test-diff review가 모두 충족된다.로 정의하는가

복구

실패를 대표 test로 추가하고 최소 변경만 허용한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

day3_review_report.md에서 어떤 status_error_code를 보는가

8차시를 마치기 전에 세 가지를 확인합니다

- 01 — 설명: 리뷰 개선은 prompt 감각이 아니라 오류 분석으로 끝냅니다이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `git diff -- src/course_services tests/test_course_services.py`
- 03 — 증거: `day3_review_report.md`과 정상·실패 test 결과가 남아 있다.

17:10-17:40 쉬는 시간 · 17:40-18:00 Q&A·실행 복구